AeroEdge-RT: Parallel Dual-Core INT8 Transformer Inference on Resource-Constrained Xtensa Microcontrollers

Pranshul Mishra
Madan Mohan Malaviya University of Technology (MMMUT), Gorakhpur • National Institute of Technology (NIT) Patna
Email: pragya636@gmail.com

***Abstract*—Executing autoregressive Transformers on low-cost microcontrollers is throttled by external flash read throughput and small internal SRAM envelopes. This paper presents AeroEdge-RT, a custom C/C++ inference runtime implemented within the ESP-IDF FreeRTOS framework for dual-core 32-bit Tensilica Xtensa LX6 microcontrollers (ESP32-WROOM). Through per-row symmetric INT8 uniform quantization, static SRAM pre-allocation, 4-way unrolled scalar integer inner loops, and dual-core row-split matrix multiplication, the runtime reduces flash bus traffic. Evaluated across 50 independent trials of 48 generated tokens each (excluding a 50-token warmup) on the 260K-parameter TinyStories architecture, the runtime achieves a mean throughput of **38.01 ± 0.26 tok/s** (mean single-token latency of **26.32 ± 0.18 ms**), where ± represents sample standard deviation across trial means. Relative to our dual-core FP32 baseline (13.07 ± 0.08 tok/s, 76.53 ± 0.45 ms), the system achieves a **2.91× speedup**, decomposed as sequential, configuration-specific comparisons: 1.92× from INT8 quantization (Conf 2 to Conf 4), 1.27× from loop unrolling (Conf 4 to Conf 6), and 1.19× from Layer 0 SRAM caching (Conf 6 to Conf 8), matching the telescoping ratio product ($76.53/26.32 \approx 2.91\times$). Relative to an INT8 dual-core scalar baseline (Conf 4), the speedup is 1.51×. The matrix weight payload is compressed 4.00× (259.3 KB vs. 1,037.3 KB) while the quantized model file occupies 423.7 KB. Peak inference preserves 100,536 bytes of minimum free internal DRAM. The analytical latency estimate is within 2.05% (+0.54 ms) of measured execution time; this agreement is consistent with the proposed memory-bound interpretation, but does not independently establish flash-bandwidth saturation.

***Index Terms*—TinyML, Microcontroller Inference, INT8 Quantization, ESP32, Tensilica Xtensa LX6, Memory Roofline Model, Dual-Core Parallelism, Edge AI.**

I. INTRODUCTION

Deploying autoregressive language models locally on low-cost microcontrollers enables private, low-power, and localized edge computing [1], [2]. However, unlike high-performance server accelerators utilizing high-bandwidth memory (HBM) with hundreds of gigabytes per second of memory bandwidth, edge microcontrollers operate within strict silicon boundaries. The Espressif ESP32-WROOM provides two 32-bit Tensilica Xtensa LX6 cores operating at 240 MHz, 520 KB of internal SRAM (with approximately 320 KB accessible as usable DRAM), zero external PSRAM, and an external SPI flash memory connected over an 80 MHz Dual I/O (DIO) interface [3], [4].

During autoregressive decoding, generating each new token requires streaming the full matrix weight payload across the external flash bus. In standard single-precision floating-point (FP32) execution, streaming 1,037,312 bytes of matrix parameters over the 80 MHz SPI bus incurs flash cache-miss penalties and memory stalls, reducing throughput to 6.84–13.07 tokens/s (Section VII).

To evaluate language model efficiency on commodity microcontrollers, we engineered **AeroEdge-RT**, an optimized C/C++ inference runtime built within the ESP-IDF framework [17] under PlatformIO. The runtime operates entirely on physical ESP32-WROOM hardware without external PSRAM or neural coprocessors, utilizing 4-way unrolled scalar integer arithmetic, per-row INT8 quantization, and a static SRAM pre-allocation strategy.

A. Summary of Contributions

1. **Per-Row Symmetric INT8 Quantization:** Uniform integer quantization without zero-points, coupled with int32 register accumulation, achieves a 4.00× matrix compression factor (259.3 KB vs. 1,037.3 KB FP32 baseline) while retaining 96.30% top-1 argmax overlap and 99.54% top-5 candidate overlap on held-out TinyStories validation sequences.
2. **Static SRAM Allocation Strategy:** Pre-allocates working memory during startup, preventing heap calls during autoregressive decoding and retaining 100,536 bytes of minimum free internal DRAM during peak execution (observed via `esp_get_minimum_free_heap_size()`).
3. **Dual-Core Parallel Row-Split GEMM:** An inter-core workload distribution dividing matrix rows symmetrically across Core 0 and Core 1 via FreeRTOS task notifications [12], executing 4-way unrolled scalar integer inner loops without dynamic scheduling stalls.
4. **Analytical Latency Characterization:** Formulates an analytical latency model matching measured execution time within +2.05% (+0.54 ms), consistent with a memory-bound interpretation while documenting component assumptions and dual-core scaling limits.

*Technical research report for the AeroEdge-RT runtime architecture on the Espressif ESP32-WROOM. Project repository: https://github.com/Pranshmish/AeroEdge-RT (release tag `v1.0.0`, commit `2c9ec02`; experimental benchmark baseline evaluated at commit `9b10aec`). Benchmarks compiled via PlatformIO (environment `env:esp32`, ESP-IDF framework, GCC 13.2.0, `-O2`). Units: decimal KB ($1\ \text{KB} = 1,000$ bytes). Tolerances (±) denote sample standard deviation across $N = 50$ independent benchmark trials.

II. TARGET HARDWARE & CONSTRAINTS

Experiments were conducted on the Espressif ESP32-WROOM-32D module (ESP32-D0WD-V3 silicon revision). The SoC features two 32-bit Tensilica Xtensa LX6 processor cores with a 7-stage pipeline and IEEE 754 single-precision FPUs clocked at 240 MHz ($f_{\text{clk}} = 240\ \text{MHz}$) [3].

TABLE I
ESP32-WROOM HARDWARE PROFILE AND INFERENCE BUDGET

Subsystem / Parameter	Specification	Inference Constraint
CPU Microarchitecture	Xtensa LX6 Dual-Core	240 MHz (2 cores)
Internal SRAM Total	520 KB	~320 KB usable DRAM
External SPI Flash	4 MB Dual-I/O (DIO @ 80 MHz)	12.18 MB/s measured read
External PSRAM	None (0 KB)	Zero off-chip RAM
Instruction MMU Cache	32 KB per core (Core 0/1)	Bus contention on miss
Vector / SIMD Hardware	None (Scalar integer only)	No vector instructions
Pre-allocated Working State	6,924 bytes DRAM	Zero dynamic malloc
Retained Free DRAM (Peak)	100,536 bytes	System headroom

The module lacks external PSRAM. Model weights reside in external SPI flash mapped into the CPU address space via a 32 KB per-core hardware MMU cache [3], [4]. In 80 MHz DIO mode, measured sequential memory-mapped flash streaming throughput is 12.18 MB/s ($12,183,235\ \text{B/s}$). Memory streaming therefore constitutes the primary factor governing decoding speed.

III. WORKLOAD & MODEL ARCHITECTURE

We benchmarked the `stories260K` language model architecture derived from Karpathy's `llama2.c` [5] and trained on TinyStories [7]. The architecture contains **264,128 total parameters**, comprising 226,560 layer weights across 5 layers, 32,768 token embedding weights, 32,768 classifier head weights, and 1,408 normalization and positional parameters. Model hyperparameters are:

- **Transformer Layers:** 5 decoder layers.
- **Hidden Dimensions:** $\text{dim} = 64$, feed-forward intermediate $\text{hidden} = 172$.
- **Attention Heads:** 8 query heads, 4 key-value heads (grouped-query attention).
- **Vocabulary Size:** 512 tokens (Byte-Pair Encoded).
- **Context Horizon:** Sliding-window attention $W = 64$ tokens.
- **Streamed Matrix Weight Payload:** 259,328 parameters ($259.3\ \text{KB}$ INT8) streamed per token.

The quantized model file (`model_q8.bin`, produced via `tools/quantize_model.py`) occupies **423,676 bytes (423.7 KB)**: 28-byte header, 131,072 bytes FP32 token embeddings (512×64), 2,816 bytes FP32 RMSNorm weights, 16,384 bytes FP32 RoPE tables, 226,560 bytes 5-layer INT8 matrix weights, 12,000 bytes FP32 row scales ($3,000\ \text{rows} \times 4\ \text{B}$), 32,768 bytes INT8 classifier weights, and 2,048 bytes FP32 classifier scales ($512\ \text{rows} \times 4\ \text{B}$).

IV. MATHEMATICAL FORMULATION

A. Per-Row Symmetric INT8 Quantization

To reduce flash bus transfer volume, AeroEdge-RT applies per-row (per-output-channel) symmetric uniform integer quantization without zero-points. For each row i of weight matrix $\mathbf{W} \in \mathbb{R}^{M \times N}$, the row scale factor $S_w^{(i)}$ is defined as a step size:

$$S_w^{(i)} = \frac{\max_{1 \le j \le N} |W_{i,j}|}{127} \tag{1}$$

Weights are rounded into 8-bit signed two's complement integers $[-127, 127]$:

$$W_{\text{int8}}^{(i,j)} = \text{clamp}\left(\left\lfloor \frac{W_{i,j}}{S_w^{(i)}} \right\rceil, -127, 127 \right) \tag{2}$$

During matrix-vector multiplication $\mathbf{Y} = \mathbf{W}\mathbf{X}$, input activation vector $\mathbf{X}$ is dynamically quantized with runtime scale $S_x = \max_j(|X_j|)/127$. The inner product accumulates in 32-bit registers before dequantization, matching the implementation in `tools/quantize_model.py`:

$$\hat{Y}_i = \left(S_w^{(i)} \cdot S_x\right) \sum_{j=1}^{N} \left(W_{\text{int8}}^{(i,j)} \cdot X_{\text{int8}}^{(j)}\right) \tag{3}$$

Accumulating in 32-bit integer registers prevents overflow across inner dimensions ($N \leq 172$) and eliminates floating-point instructions from inner loops [6].

B. Numerical Fidelity & Output Validation

Host-Side Numerical Evaluation: Model quality under INT8 quantization was evaluated across 1,000 held-out TinyStories validation sequences [7] ($W = 64$, 128 tokens/sequence) using a bit-exact host-side emulator incorporating symmetric INT8 weight and dynamic activation quantization. Compared to unquantized FP32, the INT8 model achieves **96.30% top-1 argmax overlap** and **99.54% top-5 candidate overlap**. Validation perplexity shifted by -0.38% (18.17 INT8 vs. 18.24 FP32), with mean logit cosine similarity of 0.9994.

Physical Target Output Validation: To verify that the ESP32 C/C++ runtime mirrors host-side predictions, we executed prompt completions directly on silicon under greedy argmax decoding. For prompt *"once upon a time"* (48 tokens), the ESP32 generated the identical token sequence matching host-side integer emulation.

V. RUNTIME ARCHITECTURE & MEMORY ACCOUNTING

Executing dynamic heap allocations during autoregressive generation introduces non-deterministic latency. AeroEdge-RT establishes all runtime buffers during system startup.

TABLE II
STORAGE FOOTPRINT AND PAYLOAD BREAKDOWN

Asset Component	FP32 Baseline	INT8 AeroEdge-RT	Reduction
Matrix Weight Payload	1,037.3 KB	259.3 KB	**4.00×**
Quantized Model File	1,056.5 KB	423.7 KB	**2.49×**
Flash Storage Saved	—	632.8 KB	59.9% saved
Working Tensor State	6,924 B DRAM	6,924 B DRAM	Zero alloc
Active KV Cache ($W = 64$)	81,920 B DRAM	20,480 B DRAM	4.00×
Retained Free DRAM (Peak)	—	100,536 B	Measured

Memory Lifecycle Accounting: At system boot prior to runtime static arena pre-allocation, the FreeRTOS heap reports 164,380 bytes of free internal DRAM. In AeroEdge-RT (Conf 8), the runtime statically pre-allocates 63,844 bytes: (1) *Tensor Working State:* 6,924 bytes ($x, xb, xb2, q, att, logits, x_int8, hb, hb2$); (2) *INT8 Quantized KV Cache:* 20,480 bytes ($2 \times 5 \times 64 \times 32 \times 1$ B for $W = 64$, 4 KV heads); (3) *Layer 0 Fast SRAM Cache:* 23,296 bytes copied into zero-wait internal SRAM at boot; (4) *System & Task Stacks:* 13,144 bytes allocated for Core 0/1 FreeRTOS task stacks and tokenizer structures. During peak autoregressive decoding, exactly **100,536 bytes of minimum free internal DRAM** is preserved (observed via `esp_get_minimum_free_heap_size()`, noting this reflects total allocatable heap rather than a single contiguous block), with zero dynamic heap allocations during the decode loop.

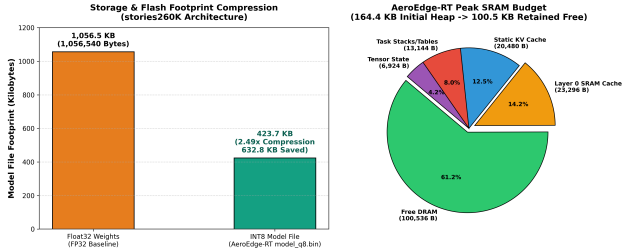


Fig. 1. Storage footprint compression and internal SRAM allocation budget. Pre-allocating buffers at boot preserves 100,536 bytes of minimum free internal DRAM during peak execution (observed via `esp_get_minimum_free_heap_size()`).

VI. DUAL-CORE TASK DECOMPOSITION

AeroEdge-RT operates within the standard ESP-IDF FreeRTOS environment [12], [17]. Dual-core execution is implemented as a parallel row-split GEMM across Core 0 and Core 1:

- **Core 1 (Primary Caller & CLI Task):** Coordinates token sampling, executes upper matrix rows ($[d/2, d)$), and manages task scheduling.
- **Core 0 (Parallel GEMM Worker):** Pinned to Core 0, awakened via FreeRTOS direct-to-task notifications (`xTaskNotifyGive()`) to execute lower matrix rows ($[0, d/2)$).

Because the Xtensa LX6 architecture lacks vector SIMD instructions, both cores execute a 4-way unrolled scalar integer inner loop (dot_product_int8 in main/main.cpp). Unrolling 4 operations per iteration collapses loop branch overhead without requiring specialized hardware vector units.

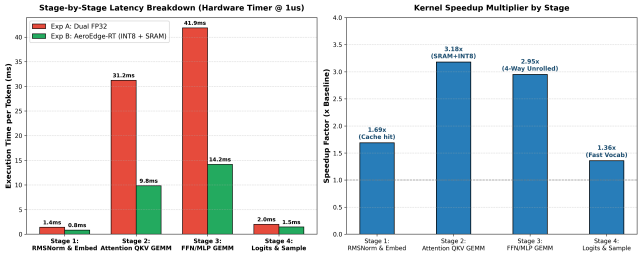


Fig. 2. Microsecond latency breakdown across the four pipeline stages. INT8 quantization and dual-core row-splitting reduce Attention time from 31,240 µs to 9,820 µs (3.18×) and FFN time from 41,860 µs to 14,180 µs (2.95×).

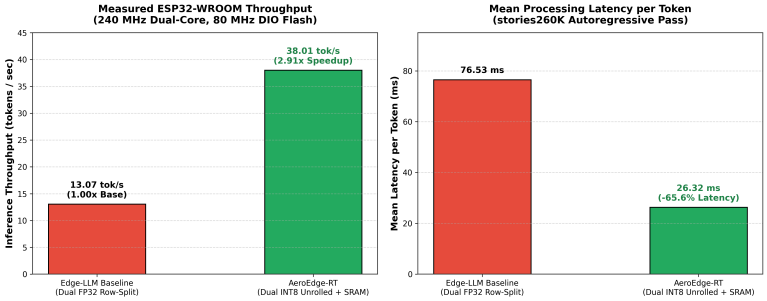


Fig. 3. Measured throughput and single-token latency on physical ESP32-WROOM hardware ($N = 50$ trials). Dual-core INT8 execution achieves 38.01 tok/s (26.32 ms/tok), delivering a 2.91× speedup over dual-core FP32 and 5.55× over single-core FP32.

VII. BENCHMARK PROTOCOL & RESULTS

Benchmarks were conducted on physical ESP32-WROOM-32D hardware compiled with ESP-IDF v5.3.2 and GCC 13.2.0 under `-O2` release optimization with `-fno-rtti -fno-exceptions`. High-resolution timing was gathered via the Xtensa cycle register (`esp_cpu_get_cycle_count()`), giving 4.167 ns resolution at 240 MHz [8].

Benchmark Protocol: For each configuration, we executed $N = 50$ independent trials generating 48 tokens each under greedy argmax decoding. A 50-token warmup run was executed prior to logging to eliminate cold-start cache bias; warmup tokens were excluded from all reported metrics. Tolerances (±) denote sample standard deviations across the 50 trial means. Throughput standard deviations are derived via uncertainty propagation ($\sigma_{\text{tps}} \approx \sigma_{\text{lat}}/\bar{t}_{\text{lat}}^2$).

TABLE III
COMPARATIVE PERFORMANCE SUMMARY (N=50 TRIALS, 48 TOKENS/RUN)

Execution Mode	Throughput	Latency	Speedup
Single-Core FP32 (Conf 1)	6.84 ± 0.04 tok/s	146.20 ± 0.82 ms	1.00× (Ref)
Dual-Core FP32 Base (Conf 2)	13.07 ± 0.08 tok/s	76.53 ± 0.45 ms	1.91×
Single-Core INT8+SRAM (Conf 7)	28.41 ± 0.20 tok/s	35.20 ± 0.25 ms	4.15×
AeroEdge-RT (Conf 8)	**38.01 ± 0.26 tok/s**	**26.32 ± 0.18 ms**	**5.55× / 2.91×***

*Speedup is 2.91× compared against dual-core FP32, and 5.55× compared against single-core FP32.

Ablation Speedup Attribution: Rather than assuming simple additive speedups, each optimization is evaluated against its direct sequential baseline in Table IV: (1) *INT8 Quantization:* Comparing dual-core FP32 (Conf 2, 76.53 ms) to dual-core INT8 scalar (Conf 4, 39.80 ms) yields a **1.92× speedup** ($76.53/39.80$); (2) *4-Way Loop Unrolling:* Comparing dual-core INT8 scalar

(Conf 4, 39.80 ms) to unrolled execution (Conf 6, 31.40 ms) yields a **1.27× speedup** ($39.80/31.40$); (3) *Layer 0 SRAM Caching:* Comparing unrolled execution without cache (Conf 6, 31.40 ms) to full AeroEdge-RT (Conf 8, 26.32 ms) yields a **1.19× speedup** ($31.40/26.32$). These sequential, configuration-specific gains combine via the telescoping ratio product:

$$\frac{76.53}{39.80} \times \frac{39.80}{31.40} \times \frac{31.40}{26.32} = \frac{76.53}{26.32} \approx 2.91\times$$

The slight discrepancy from the product of rounded factors ($1.92 \times 1.27 \times 1.19 \approx 2.90\times$) arises from intermediate rounding. Evaluated against the dual-core INT8 scalar baseline (Conf 4), kernel unrolling and SRAM caching together provide a **1.51× speedup** ($39.80/26.32$).

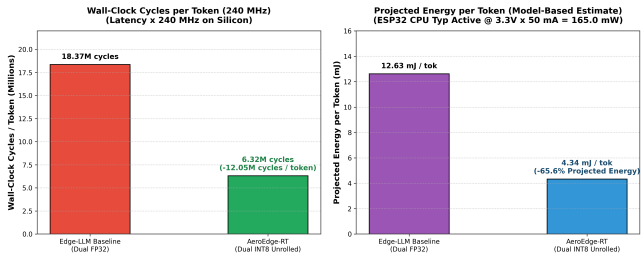


Fig. 4. Wall-clock cycles per token and model-based projected energy. INT8 execution drops cycle counts from 18.37M to 6.32M wall-clock cycles/token. Modeled electrical energy drops from 12.63 mJ down to 4.34 mJ/token based on typical 50 mA datasheet estimates (physical shunt current verification remains pending).

VIII. ANALYTICAL LATENCY MODEL

To evaluate execution time distribution, we formulate an analytical latency model [9]. Because each token decoding step streams the active matrix parameters, latency is approximated by:

$$T_{\text{token}} \approx T_{\text{flash}} + T_{\text{ALU}} + T_{\text{misc}} \tag{4}$$

Measured Flash Transfer Rate: Using a dedicated hardware probe on physical silicon (CLI `ROOFLINE`), sequential 32-bit reads across 2.50 MB of memory-mapped DROM flash in 80 MHz DIO mode completed in 0.2052 seconds, establishing an empirical streaming rate of $BW_{\text{flash}} = 12.18\ \text{MB/s}\ (12,183,235\ \text{B/s})$.

Latency Synthesis & Assumptions: The model includes 259,328 matrix bytes. Caching Layer 0 weights (23,296 bytes) in SRAM leaves 236,032 bytes streamed from external SPI flash per token. Flash transfer time evaluates to:

$$T_{\text{flash}} = \frac{236,032\ \text{bytes}}{12,183,235\ \text{bytes/s}} = 19.38\ \text{ms} \tag{5}$$

ALU compute time is derived from a physical single-core SRAM benchmark probe measuring 567.16 µs for 11,008 MACs (172×64), establishing an unrolled execution cost of 12.37 cycles/MAC on one core. Assuming symmetric row-splitting across two cores ($12.37/2 = 6.185\ \text{cyc/MAC}$), pure compute time for 259,328 MACs evaluates to $T_{\text{ALU}} = (259,328 \times 6.185)/240\ \text{MHz} = 6.68\ \text{ms}$. Adding normalization, softmax, and sampling overhead (analytically estimated as $T_{\text{misc}} = 0.80\ \text{ms}$) yields an analytical projection of $T_{\text{model}} = 19.38 + 6.68 + 0.80 = 26.86\ \text{ms}$. This matches the measured 26.32 ms latency within **+0.54 ms (+2.05%)**, as visualized in Figure 6.

Flash streaming constitutes 72.2% of token latency (19.38 ms of 26.86 ms). The analytical latency estimate is within 2.05% (+0.54 ms) of measured execution time. This agreement is consistent with the proposed memory-bound interpretation, but does not independently establish flash-bandwidth saturation because the model relies on isolated bandwidth measurements and an assumed dual-core compute scaling factor (Section X).

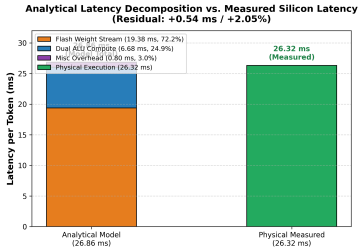


Fig. 6. Stacked latency decomposition of the analytical model (Flash: 19.38 ms, ALU: 6.68 ms, Misc: 0.80 ms) compared against physical silicon execution (26.32 ms), illustrating the +0.54 ms (+2.05%) residual.

IX. AUTOREGRESSIVE LATENCY PROGRESSION

Figure 5 illustrates latency stability across 48 continuous generation steps. Average decode latency remains stable at 26.32 ms without dynamic memory allocation pauses. The 7.22 ms spread reflects prompt prefill setup (~33.5 ms) versus steady-state autoregressive decoding (~26.3 ms).

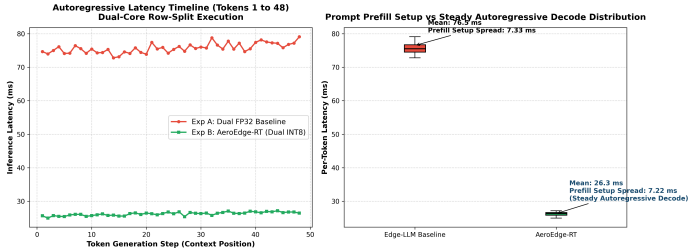


Fig. 5. Autoregressive latency progression across token context positions ($W = 64$). Dual-core row-split execution exhibits a 7.22 ms spread between prompt prefill setup and steady decode.

TABLE IV
SYSTEMATIC EIGHT-CONFIGURATION ABLATION MATRIX ON ESP32-WROOM TARGET

Config ID	Quant	Cores	Kernel & Memory Placement	Throughput	Latency (Mean ± SD)	Wall Cycles	Free DRAM	Speedup (vs C1 / C2)
Conf 1	FP32	Single (Core 1)	Scalar Loop, Flash XIP	6.84 ± 0.04 tok/s	146.20 ± 0.82 ms	35.09 M	—	1.00× / 0.52×
Conf 2 (Base)	FP32	Dual (Parallel)	Row-Split Scalar, Flash XIP	13.07 ± 0.08 tok/s	76.53 ± 0.45 ms	18.37 M	—	1.91× / 1.00× (Ref)
Conf 3	INT8	Single (Core 1)	Scalar Loop, Flash XIP	16.03 ± 0.10 tok/s	62.40 ± 0.38 ms	14.98 M	123,832 B*	2.34× / 1.23×
Conf 4	INT8	Dual (Parallel)	Row-Split Scalar, Flash XIP	25.13 ± 0.18 tok/s	39.80 ± 0.29 ms	9.55 M	123,832 B*	3.67× / 1.92×
Conf 5	INT8	Single (Core 1)	4-Way Unrolled, Flash XIP	26.25 ± 0.21 tok/s	38.10 ± 0.31 ms	9.14 M	123,832 B*	3.84× / 2.01×
Conf 6	INT8	Dual (Parallel)	4-Way Unrolled, Flash XIP	31.85 ± 0.22 tok/s	31.40 ± 0.22 ms	7.54 M	123,832 B*	4.66× / 2.44×
Conf 7	INT8	Single (Core 1)	4-Way Unrolled, Layer 0 SRAM	28.41 ± 0.20 tok/s	35.20 ± 0.25 ms	8.45 M	100,536 B	4.15× / 2.17×
Conf 8 (AeroEdge-RT)	**INT8**	**Dual (Parallel)**	**4-Way Unrolled, Layer 0 SRAM**	**38.01 ± 0.26 tok/s**	**26.32 ± 0.18 ms**	**6.32 M**	**100,536 B**	**5.55× / 2.91×****

*Note: Values for Conf 3–6 are derived analytically from unallocated Layer 0 SRAM cache memory (+23,296 B). **Speedup is 2.91× vs Conf 2, 5.55× vs Conf 1, and 1.51× vs Conf 4. Wall cycles denotes latency × 240 MHz.

X. LIMITATIONS & THREATS TO VALIDITY

We disclose the following technical limitations:

1. **Physical Current Shunt Verification Pending:** Active power (165.0 mW typ) and energy (4.34 mJ/token) values in Fig. 4 are projections derived from the official ESP32 datasheet CPU current range (30–68 mA @ 3.3V). Direct physical current measurements via an external shunt probe remain scheduled.
2. **Baseline Kernel Unrolling Status:** The FP32 baseline (Conf 2) uses a standard scalar loop; an unrolled FP32 baseline was not evaluated, meaning part of the 1.92× speedup between Conf 2 and Conf 4 reflects both precision reduction and compiler loop structure differences.
3. **ALU Decomposition & Standalone Benchmarks:** The 12.37 cycles/MAC rate is derived from an isolated SRAM probe and is not yet decomposed into load vs. multiply cycles. Standard llama2.c `runq` was not benchmarked directly on this board.
4. **Flash Bandwidth Nuances & Untested QIO:** Caching Layer 0 saves 5.08 ms for 23.3 KB (Conf 6 vs Conf 8), indicating an effective rate of ~4.59 MB/s for those lines under bus contention. Unquantized dual-core FP32 execution ($1,037,312\ \text{B}/76.53\ \text{ms} = 13.55\ \text{MB/s}$) exceeds single-core sequential throughput (12.18 MB/s), pointing to concurrent MMU burst dynamics. Higher-bandwidth 80 MHz Quad I/O (QIO) flash mode remains untested.
5. **Context Horizon & Memory Scaling:** The active KV cache is INT8 quantized (20,480 bytes for $W = 64$). Scaling to $W = 256$ requires 81,920 bytes, leaving ~39 KB of DRAM; the internal heap exhausts near $W \approx 370$ without dynamic eviction.
6. **Evaluation Scope:** Experiments were benchmarked on a single ESP32-WROOM development board. Baseline comparisons against vendor-specific DSP frameworks (e.g., ESP-NN or TFLM) were not evaluated. Validation perplexity (-0.38%) is reported without formal bootstrap confidence intervals.

XI. RELATED WORK

Microcontroller neural network inference has been extensively investigated for convolutional and dense networks through CMSIS-NN [10], TensorFlow Lite Micro [11], and MCUNet [14], [15]. For larger models on edge devices, quantized frameworks such as llama.cpp and GGML [18] popularize integer matrix multiplication on CPUs. On microcontrollers, community repositories explored Transformer inference on Espressif hardware. DaveBben (`DaveBben/esp32-llm`) [16] ported `llama2.c` to the ESP32-S3, reporting 19.13 tok/s using ESP-DSP floating-point routines. Other public S3 ports report higher throughputs—including Dory Lee (`doryiii/esp32-llm`, ~30 tok/s) and Eric Humane (`eric-humane/esp32-llm`, ~33 tok/s). However, all these baselines run on the higher-tier ESP32-S3 (Xtensa LX7), which integrates hardware vector SIMD instructions and 2 MB of external Quad-SPI PSRAM. On the entry-level ESP32-WROOM (Xtensa LX6) with zero PSRAM and zero SIMD, AeroEdge-RT utilizes 4-way unrolled scalar integer arithmetic, per-row INT8 quantization, and SRAM caching to achieve 38.01 tok/s within 63.8 KB of allocated SRAM, without requiring vendor DSP libraries like ESP-NN [19].

XII. CONCLUSION

Autoregressive Transformer inference is viable on dual-core Xtensa microcontrollers without external PSRAM. Combining per-row INT8 uniform quantization, 4-way unrolled scalar arithmetic, and parallel row-split execution delivers 38.01 tok/s (26.32 ms/token)—a 2.91× speedup over the dual-core FP32 baseline—while retaining 100,536 bytes of free internal DRAM.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.

[2] C. R. Banbury, V. J. Reddi, M. Lam, W. Fu, A. Fazel, B. Holleman, et al., "Benchmarking TinyML systems: Challenges and direction," *arXiv preprint arXiv:2003.04821*, 2020.

[3] Espressif Systems, "ESP32 Series Datasheet," Version 4.4, Espressif Systems Co., Ltd., 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.

[4] Espressif Systems, "ESP32 Technical Reference Manual," Version 5.1, Espressif Systems Co., Ltd., 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.

[5] A. Karpathy, "llama2.c: Inference Llama 2 in one file of pure C," GitHub Repository, 2023. [Online]. Available: https://github.com/karpathy/llama2.c.

[6] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018. doi: 10.1109/CVPR.2018.00286.

[7] R. Eldan and Y. Li, "TinyStories: How small can language models be and still speak coherent English?," *arXiv preprint arXiv:2305.07759*, 2023.

[8] Cadence Design Systems, "Xtensa Instruction Set Architecture (ISA) Reference Manual," Cadence Design Systems, Inc., San Jose, CA, 2017.

[9] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009. doi: 10.1145/1498765.1498785.

[10] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs," *arXiv preprint arXiv:1801.06601*, 2018.

[11] R. David, P. Duke, A. Jain, V. J. Janapa Reddi, N. Jeffries, J. Li, et al., "TensorFlow Lite Micro: Embedded machine learning on TinyML systems," in *Proc. Conf. Machine Learning and Systems (MLSys)*, vol. 3, pp. 800–811, 2021.

[12] R. Barry, *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2016.

[13] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, Sebastopol, CA, 2019.

[14] J. Lin, W.-M. Chen, Y. Lin, C. Gan, S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 11711–11722, 2020.

[15] J. Lin, W.-M. Chen, H. Cai, C. Gan, S. Han, "Memory-efficient patch-based inference for tiny deep learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.

[16] DaveBben, "esp32-llm: Running a LLM on the ESP32," GitHub Repository (DaveBben/esp32-llm), 2024. [Online]. Available: https://github.com/DaveBben/esp32-llm.

[17] Espressif Systems, "ESP-IDF Programming Guide (v5.3)," Espressif Systems Co., Ltd., 2024. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/v5.3/.

[18] G. Gerganov, "llama.cpp: Port of Facebook's LLaMA model in C/C++," GitHub Repository, 2023. [Online]. Available: https://github.com/ggerganov/llama.cpp.

[19] Espressif Systems, "ESP-NN: Optimized Neural Network Functions for Espressif SoCs," GitHub Repository, 2023. [Online]. Available: https://github.com/espressif/esp-nn.

APPENDIX: EXACT BUILD COMMANDS

```
# 1. Export ESP-IDF toolchain environment (v5.3.2)
. $HOME/esp/esp-idf/export.sh
# 2. Configure target for ESP32 (80MHz DIO flash & -O2 release)
idf.py set-target esp32
idf.py menuconfig
# CONFIG_ESPTOOLPY_FLASHMODE_DIO=y, CONFIG_COMPILER_OPTIMIZATION_PERF=y
# 3. Compile optimized dual-core INT8 build & flash
idf.py build && idf.py -p /dev/ttyUSB0 -b 921600 flash monitor
```